HW7 PDF Writeup

Step 1:

The step 1 output was:

“
Step 1: Pi= 3.14159285 in time on 1 threads: 0.12171
Step 1: Pi= 4.43253356 in time on 2 threads: 0.35916
Step 1: Pi= 4.23455290 in time on 4 threads: 0.80721
Step 1: Pi= 6.05685091 in time on 8 threads: 1.79631
Step 1: Pi= 4.91180492 in time on 12 threads: 2.44769
“

It does not compute the correct output except at 1 thread. As the number of threads increases, the estimate worsens. Time is seen to get worse with the number of threads. This is because “#pragma omp parallel” around the loop causes every thread to run the entire loop and update the same quarterpi, which causes duplicate work.

Step 2:

The step 2 output was:

“
Step 2: Pi= 3.14159285 in time on 1 threads: 0.12489
Step 2: Pi= 2.53277086 in time on 2 threads: 0.19159
Step 2: Pi= 1.10703337 in time on 4 threads: 0.20234
Step 2: Pi= 0.70518918 in time on 8 threads: 0.25307
Step 2: Pi= 0.42740461 in time on 12 threads: 0.20261
“
This result is also not correct except in the first thread. Execution time does speed up as loop iteration is divided among a higher number of threads, but does not speed up for 1-thread. “#pragma omp parallel for” ensures no work is duplicated, but quarterpi still gets updated, and therefore the result is wrong because of a race condition

Step 3:

The step 3 output was:

“
Step 3: Pi= 3.14159285 in time on 1 threads: 0.12832

Step 3: Pi= 3.14159285 in time on 2 threads: 1.02846
Step 3: Pi= 3.14159285 in time on 4 threads: 1.93747
Step 3: Pi= 3.14159285 in time on 8 threads: 2.98200
Step 3: Pi= 3.14159285 in time on 12 threads: 3.64811
“

The result is correct across the threads, but the result does not speed up, and we see the time increase with a higher number of threads. The critical directive makes each update to quarterpi happen one thread at a time, which keeps the answer correct. But this also means that they wait on each other rather than working at the same time, making it slower.

Step 4:

The step 4 output was:

“
Step 4: Pi= 3.14159285 in time on 1 threads: 0.12130
Step 4: Pi= 3.14159285 in time on 2 threads: 0.06130
Step 4: Pi= 3.14159285 in time on 4 threads: 0.03257
Step 4: Pi= 3.14159285 in time on 8 threads: 0.01657
Step 4: Pi= 3.14159285 in time on 12 threads: 0.01090
“
The result is correct, and it is efficient. This is because the “reduction(+:quarterpi) clause always requires the value to be partially summed and added correctly.